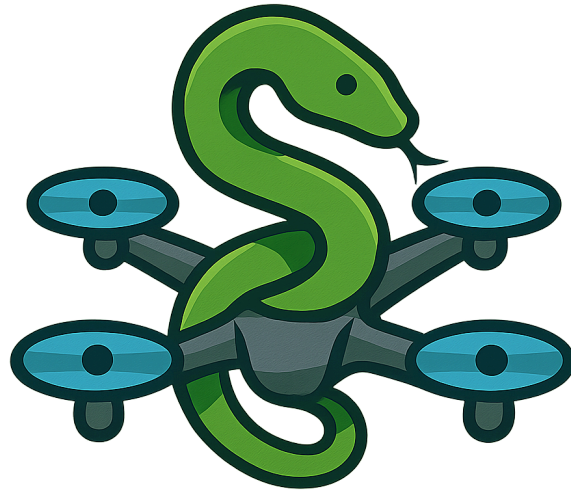


Technische Hochschule Ostwestfalen-Lippe

Bachelor of Science, Data Science

Gamification



Ausarbeitung zum Thema:

Vom Prototyp zur Spielschleife

Gamifizierung von *PythonWithDrones*

Erstellt von:	Niclas Falke, Stefan Reitemeyer und Jonas Pottmeier
Matrikelnummer:	20122542 (Niclas), 20124913 (Stefan) und 20116026 (Jonas)
Fachsemester:	4
Betrachteter Zeitraum:	10.04.2026 bis 26.07.2026
Abgabedatum:	26.07.2026
Prüfer:	Prof. Dr.-Ing. Rainer Rasche

Inhaltsverzeichnis

1	Abstrakt	1
2	Einleitung	1
3	Ausgangslage	2
3.1	Der Stand zum Modulwechsel	2
3.2	Warum das als Spiel nicht trug	2
4	Begriffliche Grundlagen	2
4.1	Gamifizierung	2
4.2	MDA	3
4.3	Selbstbestimmungstheorie	3
4.4	Flow	3
4.5	Einwände	3
5	Umsetzung	4
5.1	Überblick	4
5.2	Mehr Verben für die Drohne	5
5.2.1	Neue Befehle	5
5.2.2	Lösungsbedingungen	5
5.3	Aufgabenkurve	5
5.4	Auswertung eines Laufs	6
5.4.1	Kennzahlen	6
5.4.2	Abschlussanzeige	6
5.4.3	Terminal	7
5.5	Bestenliste	7
5.5.1	Architektur	7
5.5.2	Was gewertet wird	8
5.5.3	Betrieb ohne Backend	8
5.6	Level-Editor	9
5.6.1	Aufbau	9
5.6.2	Vom Entwurf zum Level	9
5.6.3	Einordnung	10
5.7	Reibungsabbau	10
6	Auswertung und Diskussion	10
6.1	Zuordnung zu den Grundbedürfnissen	10
6.2	Grenzen der Aussagekraft	12
6.3	Bekannte Schwächen der Umsetzung	12
6.4	Zum Risiko der Überrechtfertigung	12
7	Schlussfolgerung	13
7.1	Zusammenfassung	13
7.2	Nächste Schritte	13
	Literatur	14

1 Abstrakt

PythonWithDrones ist eine Lernanwendung, in der Python-Quelltext eine virtuelle Drohne durch dreidimensionale Level steuert. Zum Abschluss des Vorgängermoduls lag ein lauffähiger Prototyp vor, dessen Ablauf mit dem Erreichen des Zielportals endete: Das Level galt als gelöst, und damit war die Interaktion beendet. Die vorliegende Arbeit beschreibt, wie diese Anwendung im Zeitraum vom 10. April bis zum 26. Juli 2026 um Spielelemente ergänzt wurde. Hinzugekommen sind ein erweiterter Handlungsraum der Drohne mit Schieben, Aufnehmen, Abliefern und Vorausschauen, eine auf dreizehn Level verlängerte Aufgabenkurve, eine Auswertung jedes Laufs nach Abstürzen, geflogenen Feldern und Codezeilen, eine serverseitige Bestenliste mit Kontoverwaltung und Versuchsverlauf sowie ein dreidimensionaler Level-Editor, mit dem Spielende eigene Aufgaben bauen und einreichen können. Materialgrundlage der Darstellung sind 50 Commits und elf Pull Requests seit der Version 1.0.0 sowie der daraus entstandene Quelltextstand. Die Bewertung ordnet die einzelnen Elemente den Grundbedürfnissen der Selbstbestimmungstheorie zu und benennt die Stellen, an denen die Umsetzung hinter dem Entwurf zurückbleibt.

2 Einleitung

Am Ende des vorangegangenen Moduls stand eine Anwendung, die technisch funktionierte. Python lief im Browser, die Drohne bewegte sich, Level ließen sich lösen. Beim Ausprobieren mit Kommilitonen fiel jedoch etwas auf, das sich mit einem Blick in den Quelltext bestätigen ließ: Die Anwendung kannte für jedes Level genau zwei Zustände. Entweder das Zielportal war erreicht oder nicht. Ob eine Lösung mit vier Zeilen oder mit vierzig auskam, ob sie auf Anhieb saß oder erst nach zwölf Abstürzen, blieb ohne Folgen. Wer das letzte Level abgeschlossen hatte, hatte keinen Anlass, die Seite noch einmal zu öffnen.

Diese Beobachtung ist der Ausgangspunkt der vorliegenden Arbeit. Sie stellt nicht die Frage, wie sich Python im Browser ausführen lässt, denn das war Gegenstand des Vorgängermoduls und ist gelöst. Sie fragt stattdessen, welche Spielelemente sich in eine bestehende Lernanwendung einbauen lassen, welche Wirkung von ihnen strukturell zu erwarten ist und welche Kosten sie im laufenden Betrieb verursachen. Gamifizierung wird dabei nicht als Aufsatz verstanden, den man einer fertigen Anwendung überstülpt, sondern als Eingriff in die Regeln: Wenn eine Bestenliste nach Schritten sortiert, verändert das, worüber Lernende beim Schreiben ihres Programms nachdenken.

Die Arbeit stützt sich auf die Versionsgeschichte des Projekts [1]. Als Startpunkt dient das Tag v1.0.0 vom 10. April 2026, das den Stand zum Abschluss des Vorgängermoduls markiert. Alles, was danach entstanden ist, gehört in den Betrachtungsrahmen. Die Commit- und Pull-Request-Historie liefert dabei sowohl die Chronologie als auch die Begründungen, die zum jeweiligen Zeitpunkt festgehalten wurden.

Abschnitt 3 fasst den Ausgangsstand so weit zusammen, wie es zum Verständnis der späteren Eingriffe nötig ist. Abschnitt 4 führt die begrifflichen Werkzeuge ein, mit denen die Entwurfsentscheidungen anschließend beschrieben werden. Abschnitt 5 bildet den Hauptteil und geht die sechs Arbeitsstränge durch, die seit April entstanden sind. Abschnitt 6 wertet das Ergebnis aus, benennt offene Baustellen und ordnet die Elemente einem Bedürfnisraster zu. Abschnitt 7 fasst zusammen und beschreibt die nächsten Schritte.

3 Ausgangslage

3.1 Der Stand zum Modulwechsel

Die Anwendung ist ein statischer Export eines Next.js-Projekts und wird über GitHub Pages ausgeliefert. Sie benötigte zu diesem Zeitpunkt keine serverseitige Komponente. Geschriebener Quelltext wird an einen Web Worker übergeben, in dem Pyodide einen zu WebAssembly übersetzten CPython-Interpreter betreibt [2]. Jeder Drohnenebefehl erzeugt dort ein Datenobjekt, das über `postMessage` an den Hauptthread zurückgereicht wird. Dort nimmt eine Warteschlange die Objekte entgegen und arbeitet sie nacheinander als Animationen in einer mit React Three Fiber aufgebauten Szene ab [3]. Der Fortschritt liegt im `localStorage` des Browsers.

Level sind YAML-Dateien. Sie beschreiben den Startpunkt der Drohne und eine Folge horizontaler Schichten, in denen jede Zelle über eine Block-Kennung belegt ist. Welche Eigenschaften eine solche Kennung hat, ob ein Block also eine Kollision auslöst oder als Ziel gilt, steht zentral in der Datei `block-registry.tsx`. Sowohl die Python-Seite als auch die Darstellung greifen auf dieselbe Quelle zu.

Zum Zeitpunkt von `v1.0.0` umfasste dieser Aufbau sechs spielbare Level, neun Blocktypen und zehn öffentliche Methoden am Drohnenobjekt. Der Handlungsraum bestand aus Bewegen, Steigen, Sinken, Drehen und zwei Abfragen, mit denen sich ein blockierter Weg und das Erreichen des Portals prüfen ließen.

3.2 Warum das als Spiel nicht trug

Aus der Perspektive des Gamification-Moduls fehlten dieser Anordnung drei Dinge.

Erstens gab es keine Bewertung. Der einzige Wert, den die Anwendung über eine Lösung kannte, war ihre Existenz. Damit fehlte jede Grundlage für Rückmeldung, die über „geschafft“ hinausgeht, und damit auch jeder Maßstab für Verbesserung.

Zweitens gab es keinen Bezug zu anderen. Der Fortschritt lag ausschließlich lokal im Browser. Zwei Personen, die dasselbe Level lösten, erfuhren nichts voneinander. Ein Wechsel des Geräts löschte den Fortschritt.

Drittens gab es keinen Spielraum. Die Inhalte lagen als YAML-Dateien im Repository, und wer eigene Aufgaben beisteuern wollte, musste die Dateistruktur kennen, das Projekt auschecken und einen Pull Request stellen. Für die Zielgruppe, also Menschen, die gerade ihre erste Schleife schreiben, ist das keine erreichbare Handlung.

4 Begriffliche Grundlagen

4.1 Gamifizierung

Deterding et al. bestimmen Gamifizierung als den Einsatz von Elementen aus dem Spieldesign in Zusammenhängen, die selbst keine Spiele sind [4]. Wichtig an dieser Formulierung ist die Beschränkung auf Elemente. Es geht nicht darum, aus einer Lernumgebung ein Spiel zu machen, sondern einzelne Bausteine zu übernehmen, deren Wirkung im Spiel bekannt ist.

Werbach und Hunter fassen die drei am häufigsten übernommenen Bausteine unter dem Kürzel PBL zusammen: Punkte, Abzeichen und Bestenlisten [5]. Punkte machen eine Leistung zählbar, Abzeichen machen sie sichtbar, Bestenlisten machen sie vergleichbar. Diese Trias ist zugleich der Teil der Gamifizierung, der am leichtesten zu bauen und am leichtesten zu missbrauchen ist.

Im vorliegenden Projekt sind Punkte und Bestenliste umgesetzt, Abzeichen bewusst nicht (siehe Abschnitt 7.2).

4.2 MDA

Das MDA-Modell trennt drei Betrachtungsebenen [6]. Mechaniken sind die Regeln und Bausteine, die eine Entwicklerin direkt schreibt. Dynamiken sind das Verhalten, das im Betrieb daraus entsteht. Ästhetik ist die Erfahrung, die dabei bei den Spielenden ankommt. Der Nutzen dieser Trennung liegt in der Richtung der Wirkung: Gebaut wird bei den Mechaniken, erlebt wird bei der Ästhetik, und dazwischen liegt eine Schicht, die sich nicht direkt festlegen lässt.

Ein Beispiel aus diesem Projekt macht das konkret. Die Mechanik ist eine Sortierreihenfolge in einer SQL-Abfrage. Die Dynamik ist, dass Spielende anfangen, ihren Lösungsweg zu kürzen, statt ihn nur lauffähig zu machen. Die Ästhetik wäre das Gefühl, einen eleganten Algorithmus gefunden zu haben. Zwischen der ersten und der dritten Ebene liegt eine Vermutung, die diese Arbeit nicht belegen kann, aber begründen muss.

4.3 Selbstbestimmungstheorie

Ryan und Deci beschreiben intrinsische Motivation als abhängig von drei Grundbedürfnissen: Kompetenzerleben, Autonomie und soziale Eingebundenheit [7]. Sailer et al. haben in einer experimentellen Studie gezeigt, dass sich einzelne Spielelemente diesen Bedürfnissen unterschiedlich klar zuordnen lassen. Punkte, Fortschrittsbalken und Rückmeldung wirkten in ihrer Untersuchung vor allem auf das Kompetenzerleben, während Avatare und geteilte Erzählungen eher die soziale Eingebundenheit ansprachen [8].

Dieses Raster dient in Abschnitt 6 als Ordnungsrahmen für die Auswertung. Es hat den praktischen Vorteil, dass es Lücken sichtbar macht: Sobald alle gebauten Elemente in dieselbe Spalte fallen, ist der Entwurf einseitig.

4.4 Flow

Csikszentmihalyi beschreibt Flow als Zustand vollständiger Vertiefung, der sich einstellt, wenn Anforderung und Fähigkeit in einem engen Verhältnis stehen [9]. Liegt die Anforderung darüber, entsteht Überforderung, liegt sie darunter, Langeweile. Für eine Lernanwendung mit fester Reihenfolge bedeutet das, dass die Steigerung zwischen zwei Levels weder zu klein noch zu groß sein darf, und dass die Anwendung sichtbar machen sollte, was ein Level verlangt, bevor es geöffnet wird.

4.5 Einwände

Gamifizierung steht seit ihrer Popularisierung unter Kritik. Bogost bezeichnet die verbreitete Form als Ausbeutungssoftware und wirft ihr vor, die Form von Spielen zu übernehmen, ohne ihre Substanz mitzuliefern [10]. Der Vorwurf trifft dort, wo Punkte und Ranglisten eine Tätigkeit belohnen sollen, die aus sich heraus keinen Reiz hat.

Empirisch ist die Lage gemischt. Die Übersichtsarbeit von Hamari et al. findet überwiegend positive Effekte, weist aber darauf hin, dass ein großer Teil der ausgewerteten Studien methodisch schwach ist und dass die Wirkung stark vom Kontext und von der Nutzergruppe abhängt [11]. Hinzu kommt der Überrechtfertigungseffekt: Die Metaanalyse von Deci et al. zeigt, dass materielle Belohnungen, die an eine Tätigkeit gekoppelt werden, die intrinsische Motivation für genau diese Tätigkeit senken können [12].

Für das vorliegende Projekt lassen sich daraus zwei Leitplanken ableiten. Erstens sollten die gemessenen Größen Eigenschaften der Lösung sein und nicht Eigenschaften des Verhaltens, das zur Lösung geführt hat. Zweitens darf keine Belohnung den Zugang zu Lerninhalten steuern. Beide Punkte werden in Abschnitt 5 an den betreffenden Stellen aufgegriffen.

5 Umsetzung

5.1 Überblick

Zwischen dem 10. April und dem 26. Juli 2026 entstanden 50 Commits in elf Pull Requests, verteilt auf drei Beitragende. Der Vergleich der Bäume von v1.0.0 und HEAD weist 113 geänderte Dateien mit 8205 hinzugefügten und 1102 entfernten Zeilen aus. Diese Arbeit lässt sich in sechs Stränge gliedern, die Tab. 1 zusammenfasst.

Strang	Kern der Änderung	Zeitraum
Handlungsraum	Schieben, Aufnehmen, Abliefern, Vorschauen; fünf neue Blocktypen; Lösungsbedingungen im Levelformat	Juli 2026
Aufgabenkurve	Sechs auf dreizehn Level, überarbeitete Einstiegsreihe, Schlagworte als Vorschau	Juli 2026
Laufauswertung	Abstürze, geflogene Felder und Codezeilen je Lauf; Abschlussanzeige; Terminal für print	Juli 2026
Bestenliste	FastAPI-Dienst mit Postgres, Konten, JWT, Rangfolge und Versuchsverlauf	Juni bis Juli 2026
Level-Editor	Dreidimensionaler Editor, Testlauf, YAML-Export, Einreichung über GitHub	Mai bis Juli 2026
Reibungsabbau	Mobile Darstellung, eigene Fehlerseite, Rückmeldeknopf, automatisierte Tests	Juli 2026

Tab. 1: Die sechs Arbeitsstränge seit Version 1.0.0.

Gemeinsam ergeben diese Stränge eine geschlossene Schleife, die es vorher nicht gab. Abb. 1 zeigt sie im Überblick. Entscheidend ist der Rücklauf: Vor der Erweiterung endete der Ablauf beim Erreichen des Ziels, jetzt führt jede Auswertung wieder auf eine mögliche nächste Handlung.

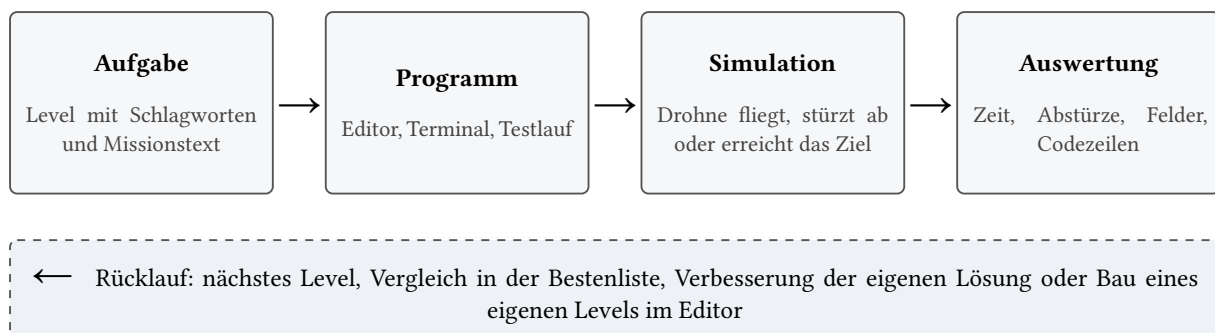


Abb. 1: Die Spielschleife nach der Erweiterung. Vor Version 1.0.0 endete der Ablauf hinter der Simulation, weil weder eine Auswertung noch ein Rücklauf existierten.

5.2 Mehr Verben für die Drohne

5.2.1 Neue Befehle

Der erste Eingriff betraf die Python-Schnittstelle selbst. Solange die Drohne nur fliegen und sich drehen konnte, gab es genau eine Art von Ziel, nämlich eine Position zu erreichen. Alle Level unterschieden sich damit nur in der Geometrie des Weges. Mit den Commits aus Pull Request 105 kamen vier Befehle hinzu, die den Zustand der Welt verändern oder abfragen:

- `drone.push()` schiebt einen als schiebbar gekennzeichneten Block um ein Feld weiter und rückt anschließend selbst nach. Ist das Feld dahinter belegt, bleibt die Aktion folgenlos und meldet den Grund.
- `drone.pickup()` und `drone.deliver()` nehmen ein Paket auf und legen es auf einer Ablagefläche wieder ab. Beide prüfen den Zustand und melden, wenn nichts aufzunehmen oder nichts abzulegen ist.
- `drone.scan(n)` liefert die Block-Kennungen der nächsten n Felder in Blickrichtung und bricht am ersten blockierenden Feld ab. Ohne Argument gibt die Methode eine einzelne Kennung zurück.

`scan` ersetzt dabei die frühere Abfrage `is_path_blocked`, die nur einen Wahrheitswert lieferte. Der Unterschied ist didaktisch bedeutsam: Eine Zeichenkette oder eine Liste zwingt dazu, das Ergebnis zu vergleichen und in eine Bedingung einzusetzen, während ein Wahrheitswert direkt in ein `if` fällt. Damit lassen sich Level bauen, in denen die Drohne unterscheiden muss, worauf sie trifft, statt nur ob sie auf etwas trifft.

Insgesamt wuchs die öffentliche Schnittstelle von zehn auf vierzehn Methoden, die Blockregistrierung von neun auf vierzehn Typen. Neu sind Münze, Kiste, Paket, Ablagefläche und Zielmarkierung.

5.2.2 Lösungsbedingungen

Damit die neuen Handlungen überhaupt Bedeutung bekommen, musste das Levelformat mitwachsen. Ein Level kann nun neben dem Erreichen des Portals verlangen, dass eine bestimmte Zahl Münzen eingesammelt, ein Paket abgeliefert oder eine Kiste auf ein markiertes Feld geschoben wurde. Die Prüfung liegt in `model.py` und läuft ab, sobald die Drohne das Portal betritt.

Der interessante Teil ist der Fall, in dem eine Bedingung offen ist. Frühere Versionen hätten hier schlicht nichts getan, und für Lernende wäre nicht unterscheidbar gewesen, ob das Programm falsch war oder die Anwendung hakte. Stattdessen sendet die Python-Seite jetzt eine eigene Nachricht vom Typ `hint`, deren Text die offenen Bedingungen benennt. Aus einem stummen Nichtereignis wird eine Erklärung.

5.3 Aufgabenkurve

Der Levelbestand stieg von sechs auf dreizehn. Ein Teil davon entstand aus einer Überarbeitung der Einstiegsreihe im Juli, bei der die ersten Level neu geordnet und ihre Beschreibungen gestrafft wurden. Die neuen Level greifen die erweiterten Befehle auf, sodass jeder Schritt der Reihe entweder ein Programmierkonzept oder eine Drohnenfähigkeit einführt, aber möglichst nicht beides gleichzeitig.

Jedes Level trägt Schlagworte, die in der Übersicht angezeigt werden. Sie nennen einerseits eine grobe Schwierigkeit und andererseits das behandelte Konzept, etwa Schleifen, Funktionen oder Bedingungen. Diese Vorschau ist der direkte Versuch, die von Csikszentmihalyi beschriebene

Passung zwischen Anforderung und Fähigkeit sichtbar zu machen, bevor jemand ein Level öffnet [9].

Gesperrte Level bleiben in der Übersicht sichtbar und tragen ein Schloss. Das ist eine bewusste Entscheidung gegen das Ausblenden: Ein sichtbares, aber noch verschlossenes Ziel ist ein Anreiz, ein unsichtbares ist keiner.

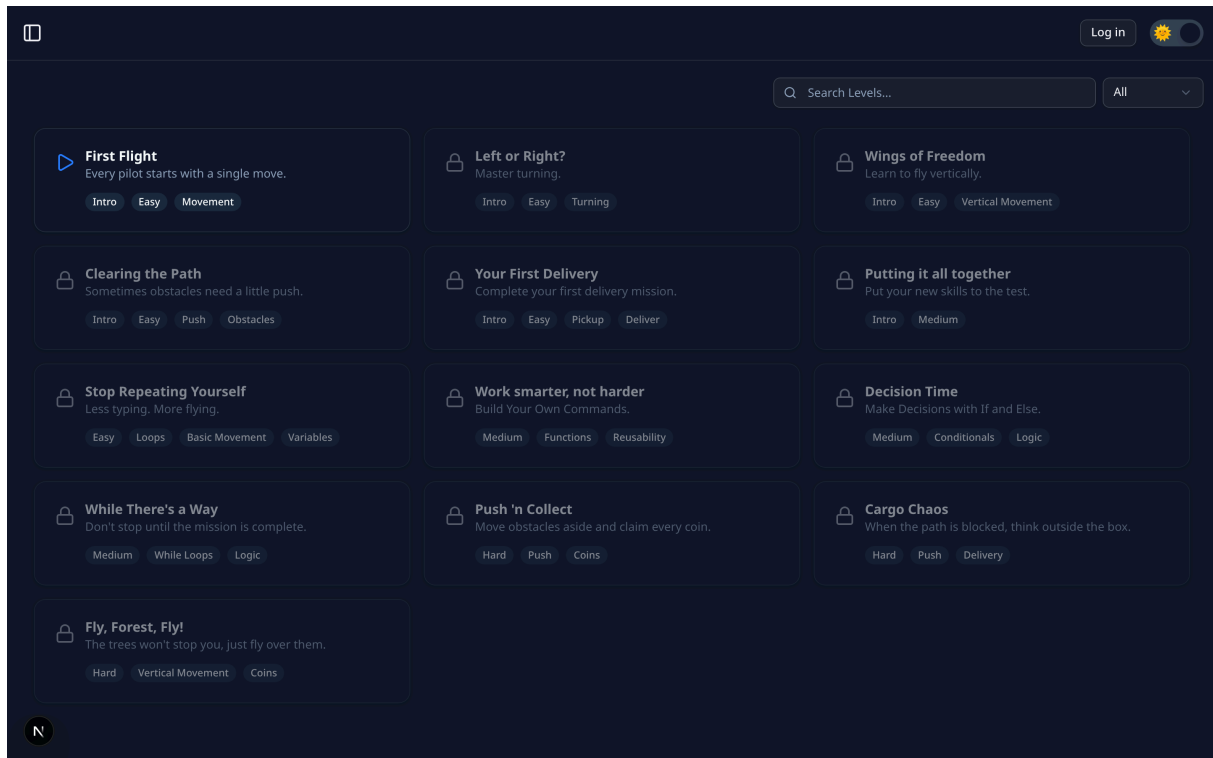


Abb. 2: Die Levelübersicht mit dreizehn Einträgen. Freigeschaltete Level tragen ein Abspielsymbol, gesperrte ein Schloss. Die Schlagworte künden Schwierigkeit und behandeltes Konzept an.

5.4 Auswertung eines Laufs

5.4.1 Kennzahlen

Die Python-Seite führt seit Juli zwei Zähler mit. `distance` steigt bei jedem erfolgreichen Ortswechsel, `crash_count` bei jeder Kollision. Beide werden über `get_stats()` an den Hauptthread gereicht. Die dritte Kennzahl, die Zahl der Codezeilen, ermittelt die Oberfläche selbst aus dem eingegebenen Text.

Diese Auswahl ist nicht zufällig. Alle drei Größen beschreiben das Programm, nicht die Person: Wie viele Felder ein Weg braucht, wie oft eine Lösung an eine Wand fliegt und wie kompakt sie formuliert ist, sind Eigenschaften des Algorithmus. Wie schnell jemand tippt, ist es nicht. Das ist die praktische Umsetzung der ersten Leitplanke aus Abschnitt 4.5.

5.4.2 Abschlussanzeige

Beim Erreichen des Ziels erscheint eine Anzeige mit der benötigten Zeit und den drei Kennzahlen, ergänzt um einen kurzen Kommentar, der sich nach der Zahl der Abstürze richtet. Ein Lauf ohne Kollision wird anders quittiert als einer mit sieben. Der Kommentar ist bewusst knapp gehalten und dient allein dazu, die nackte Zahl in eine Aussage zu übersetzen.

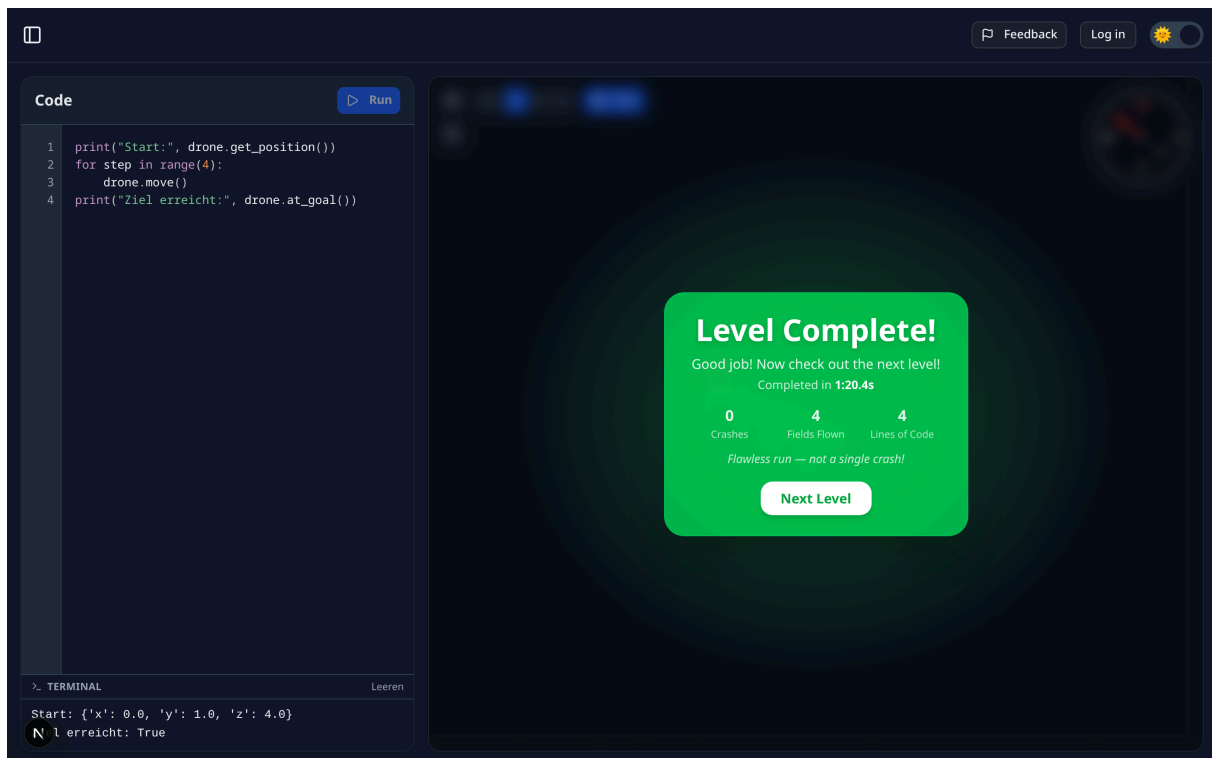


Abb. 3: Abschlussanzeige nach einem gelösten Level, im Hintergrund die Szene. Links unten ist das Terminal eingebettet, das die Ausgaben der beiden print-Aufrufe zeigt.

5.4.3 Terminal

Bis Juli lief die Ausgabe von `print` ausschließlich in die Entwicklerkonsole des Browsers. Für die Zielgruppe war sie damit praktisch nicht vorhanden, obwohl `print` das erste Werkzeug ist, mit dem Anfänger den Zustand eines Programms untersuchen. Der Worker leitet die Ausgabe jetzt zusätzlich als eigene Nachricht an den Hauptthread weiter, wo sie unter dem Editor in einem Terminalbereich erscheint. Dieselbe Bahn nutzen auch die Hinweistexte der Drohnenbefehle, etwa wenn `pickup` ins Leere greift.

Der Beitrag zur Gamifizierung liegt hier nicht in einem Spielelement, sondern in der Verkürzung der Rückmeldeschleife. Ein Fehler, dessen Ursache sichtbar ist, führt zu einem weiteren Versuch. Ein Fehler ohne sichtbare Ursache führt zum Abbruch.

5.5 Bestenliste

5.5.1 Architektur

Die Bestenliste ist der einzige Teil des Projekts, der eine serverseitige Komponente verlangt. Sie besteht aus einem FastAPI-Dienst [13] und einer Postgres-Datenbank, die zusammen mit dem Frontend über Docker Compose gestartet werden. Der Datenbankzugriff läuft asynchron über SQLAlchemy [14]. Kennwörter werden mit Argon2 gehasht, die Anmeldung liefert ein JWT, und die Anmelde- und Registrierungsrouen sind ratenbegrenzt. Zugelassene Ursprünge stehen in einer Liste in der Umgebungskonfiguration.

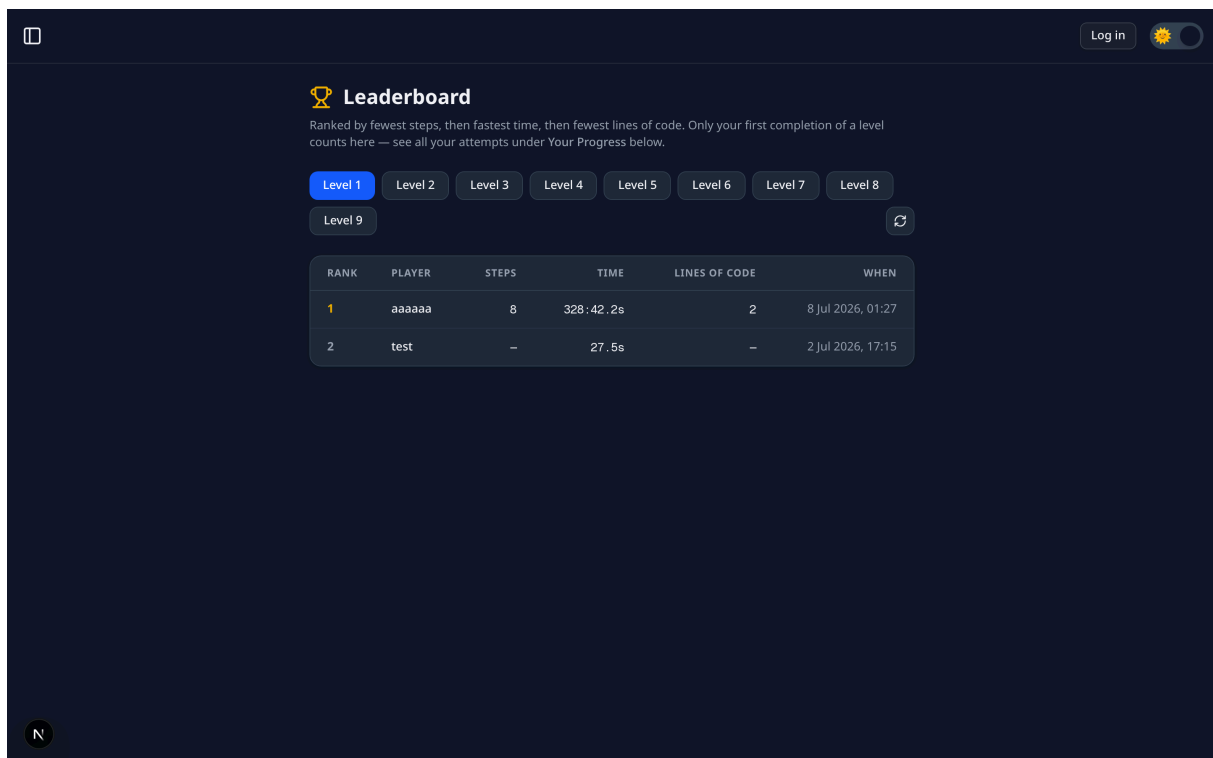
Der Dienst führt drei Tabellen. `users` hält die Konten, wobei die Eindeutigkeit des Namens über einen funktionalen Index auf der kleingeschriebenen Fassung erzwungen wird, sodass sich zwei Konten nicht allein durch Groß- und Kleinschreibung unterscheiden können. `scores` hält je Konto und Level genau einen Eintrag, nämlich die erste Lösung. `attempts` hält jeden einzelnen Versuch.

5.5.2 Was gewertet wird

Die Rangfolge ergibt sich aus drei Kriterien in dieser Reihenfolge: zuerst die Zahl der Schritte, dann die benötigte Zeit, zuletzt die Zahl der Codezeilen. Die Reihenfolge ist die inhaltlich wichtigste Entscheidung dieses Strangs. Hätte die Zeit an erster Stelle gestanden, wäre die Bestenliste eine Tippgeschwindigkeitsliste geworden, und wer die Lösung bereits kennt, hätte immer gewonnen. Die Schrittzahl dagegen ist eine Eigenschaft des gefundenen Weges. Sie lässt sich nur verbessern, indem man das Level besser versteht.

Gewertet wird ausschließlich die erste Lösung eines Levels. Das schützt die Liste davor, zu einer Anzeige der verfügbaren Freizeit zu werden, kostet aber den Anreiz, eine gefundene Lösung zu überarbeiten. Als Ausgleich hält die Tabelle `attempts` den vollständigen Verlauf, den Angemeldete unter der eigenen Fortschrittsanzeige einsehen können. Der Vergleich mit anderen ist damit einmalig, der Vergleich mit sich selbst dauerhaft.

Die Zeitmessung beginnt beim Öffnen des Levels und endet bei der ersten erfolgreichen Lösung. Sie läuft als Wanduhr im Hauptthread und ist damit unabhängig davon, mit welcher Geschwindigkeit die Animation abgespielt wird oder wie lange der Python-Interpreter für die Ausführung braucht.



RANK	PLAYER	STEPS	TIME	LINES OF CODE	WHEN
1	aaaaaa	8	328.42.2s	2	8 Jul 2026, 01:27
2	test	—	27.5s	—	2 Jul 2026, 17:15

Abb. 4: Bestenliste einer laufenden Instanz. Die Spaltenköpfe sind sortierbar. Der zweite Eintrag stammt aus der Zeit vor der Einführung der Schrittzählung und trägt deshalb Platzhalter in zwei Spalten.

5.5.3 Betrieb ohne Backend

Die Anwendung soll weiterhin ohne Server benutzbar bleiben, weil sie als statischer Export auf GitHub Pages liegt. Der API-Client liest dazu eine einzige Umgebungsvariable. Ist sie nicht gesetzt, liefert jeder Aufruf `null` zurück, und sämtliche Aufrufe der Bestenliste laufen ins Leere, ohne einen Fehler zu erzeugen. Das Spiel funktioniert dann vollständig, nur eben ohne Vergleich.

Anmeldung ist an keiner Stelle Voraussetzung für das Spielen. Der Anmeldedialog bietet neben Registrierung und Anmeldung ausdrücklich einen Gastmodus an. Damit ist die zweite Leitplanke aus Abschnitt 4.5 eingehalten: Kein Lerninhalt hängt an einer Belohnungsmechanik.

5.6 Level-Editor

5.6.1 Aufbau

Der Editor ist der umfangreichste einzelne Beitrag des Zeitraums. Er wurde im Mai begonnen und bis Juli in mehreren Durchgängen überarbeitet. Er läuft vollständig im Browser und arbeitet auf derselben Datenstruktur wie das Spiel selbst.

Die Oberfläche trennt zwei Modi. Im Setzmodus platziert oder löscht ein Klick einen Block auf der aktiven Ebene, im Kameramodus dreht und zoomt derselbe Klick die Ansicht. Da beim Bauen ständig zwischen beidem gewechselt wird, lässt sich der Kameramodus zusätzlich durch Halten der Alt-Taste vorübergehend aktivieren, das Löschen entsprechend über die X-Taste. Neben Setzen und Löschen gibt es ein Werkzeug für den Startpunkt der Drohne.

Die Ebenen werden über einen Schrittwähler durchlaufen. Sichtbar bleiben dabei alle Ebenen, die aktive wird durch ein Gitter hervorgehoben, sodass die Tiefe beim Bauen nicht verlorengeht. Ein eigenes Formular legt die Ausdehnung in allen drei Achsen fest, ein weiteres Titel, Beschreibung und Schlagworte.

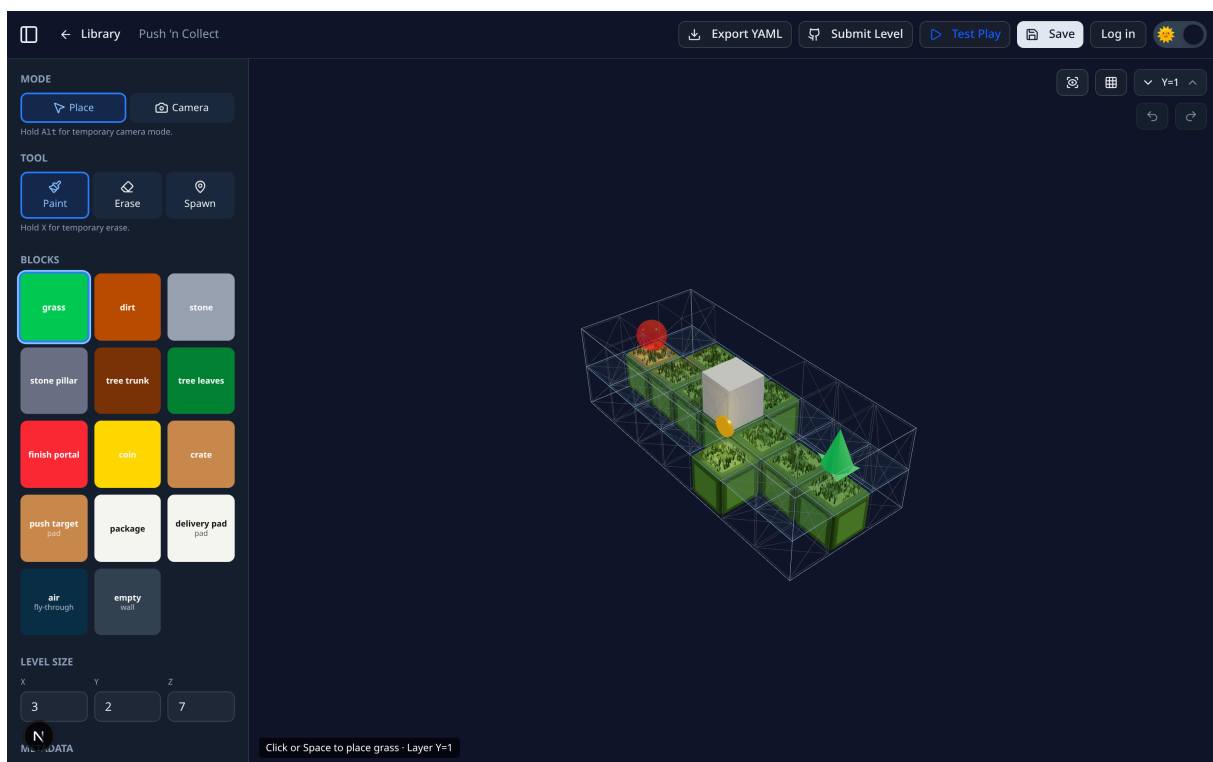


Abb. 5: Der Level-Editor mit geladener Vorlage. Links Modus, Werkzeuge, Blockpalette und Größenangaben, oben rechts der Ebenenwähler und die Rücknahme, oben die Aktionen Export, Einreichung, Testlauf und Speichern.

5.6.2 Vom Entwurf zum Level

Drei Wege führen aus dem Editor heraus. Der Testlauf öffnet das gebaute Level in einem Dialog mit vollständiger Spielumgebung, also eigenem Editor, eigener Pyodide-Instanz und eigener Szene. Der Dialog baut diese Umgebung erst beim Öffnen auf und wirft sie beim Schließen wieder weg, damit nicht dauerhaft ein zweiter Interpreter mitläuft. Der Export schreibt eine YAML-Datei in

den Download-Ordner. Die Einreichung öffnet ein vorbereitetes GitHub-Issue, in dessen Formular das erzeugte YAML bereits eingetragen ist.

Eine Entscheidung aus dem Juli verdient eigene Erwähnung. Ursprünglich mussten Autoren die Lösungsbedingungen eines Levels von Hand angeben, also etwa eintragen, dass drei Münzen einzusammeln sind. Das ging regelmäßig schief, weil Geometrie und Bedingung auseinanderliefen. Seitdem leitet der Editor die Bedingungen aus dem gebauten Level ab: Wer drei Münzen platziert, baut ein Level mit drei einzusammelnden Münzen. Die Fehlerquelle verschwindet, und der Bauvorgang wird um einen Schritt kürzer.

5.6.3 Einordnung

Der Editor adressiert das Bedürfnis nach Autonomie, das von Punkten und Bestenlisten gerade nicht bedient wird [7]. Er verschiebt die Rolle vom Lösen vorgegebener Aufgaben zum Stellen eigener. Für den Lerneffekt ist das insofern bedeutsam, als das Bauen eines lösbaren Levels verlangt, die Regeln der Simulation vollständig verstanden zu haben.

Gleichzeitig ist er eine Inhaltsquelle. Der Aufwand für ein neues Level lag vorher bei handgeschriebenem YAML und vier Stellen im Code, die synchron gehalten werden müssen. Er liegt jetzt bei einem Formular und einem Klick auf Einreichen.

5.7 Reibungsabbau

Der sechste Strang enthält Arbeiten, die für sich genommen keine Spielelemente sind, aber Voraussetzung dafür, dass die anderen fünf wirken.

Im Juli wurde die Darstellung auf schmalen Bildschirmen überarbeitet. Betroffen waren unter anderem der Ebenenwähler des Editors, der außerhalb des sichtbaren Bereichs lag, ein Mausextraktionsereignis, das nie gebunden wurde, und eine überfüllte Kopfzeile. Da ein erheblicher Teil der Zugriffe von Mobilgeräten kommt, entscheidet diese Ebene darüber, ob jemand überhaupt bis zum ersten Level kommt.

Ein zweiter Punkt betrifft ungültige Adressen. Ein Aufruf von `/level/99` führte zuvor im Entwicklungsserver zu einem Serverfehler. Seit Juli existiert eine eigene Fehlerseite mit animierter Drohne, und die Routenkonfiguration weist unbekannte Parameter sauber ab. Das ist kein Spielelement, verhindert aber, dass ein Fehlgriff wie ein Defekt der Anwendung aussieht.

Schließlich entstand im Juli eine automatisierte Testabdeckung. 118 Testfunktionen prüfen die Spiellogik in `game.py` und `model.py` sowie den Bestenlisten-Dienst. Die Spieltests täuschen dazu das Brückenmodul zur JavaScript-Seite vor, sodass der unveränderte Quelltext unter normalem CPython läuft. Die Dienst-Tests sprechen die Anwendung im selben Prozess an und arbeiten je Test gegen eine wegwerfbare SQLite-Datenbank statt gegen Postgres.

6 Auswertung und Diskussion

6.1 Zuordnung zu den Grundbedürfnissen

Tab. 2 ordnet die gebauten Elemente den drei Grundbedürfnissen zu. Die Zuordnung ist als Entwurfsargument zu lesen, nicht als Messergebnis.

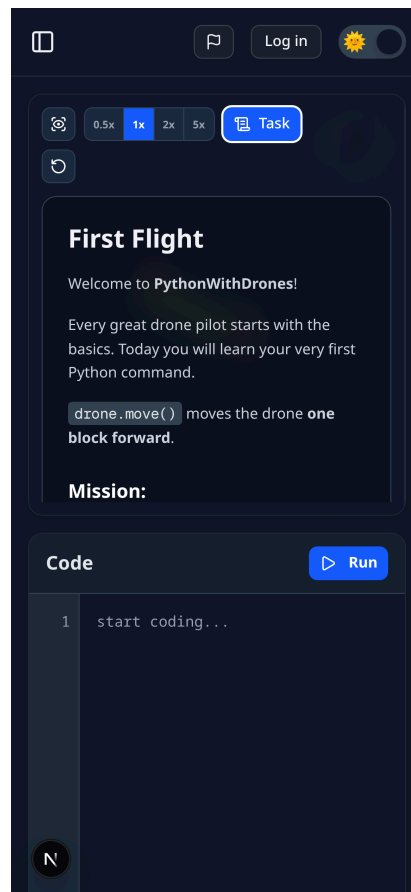


Abb. 6: Die Levelansicht auf einem schmalen Bildschirm. Aufgabentext, Szene und Editor stehen untereinander statt nebeneinander.

Bedürfnis	Umgesetzte Elemente	Begründung
Kompetenz	Kennzahlen je Lauf, Abschlussanzeige, Versuchsverlauf, gestufte Levelreihe, Terminal, Hinweise bei offenen Bedingungen	Der Fortschritt wird abgestuft statt binär sichtbar. Fehlschläge liefern eine benennbare Ursache statt eines stummen Abbruchs.
Autonomie	Level-Editor mit Testlauf, Export und Einreichung; freie Wahl unter den freigeschalteten Levels; Gastmodus ohne Konto	Spielende können eigene Inhalte erzeugen und weitergeben, statt nur vorgegebene abzuarbeiten. Kein Lerninhalt hängt an einem Konto.
Soziale Eingebundenheit	Bestenliste je Level, Konten mit sichtbarem Namen, Einreichung eigener Level über das Repository	Der bislang rein lokale Fortschritt bekommt einen Bezugsrahmen. Die schwächste der drei Spalten, siehe Abschnitt 7.2.

Tab. 2: Zuordnung der Elemente zu den Grundbedürfnissen nach Ryan und Deci [7].

Auffällig ist das Ungleichgewicht. Der Kompetenzstrang ist gut besetzt, die Autonomie über den Editor solide, die soziale Ebene dünn. Eine Bestenliste stellt Personen nebeneinander, sie bringt sie nicht miteinander in Kontakt. Sailer et al. ordnen der sozialen Eingebundenheit vor allem gemeinsame Erzählungen und Gruppenaufgaben zu [8], und beides fehlt hier vollständig.

6.2 Grenzen der Aussagekraft

Die wichtigste Einschränkung ist methodisch. Es wurde keine Nutzerstudie durchgeführt. Alle Aussagen dieser Arbeit über Wirkung sind Entwurfsargumente auf Grundlage der zitierten Literatur, keine Belege. Angesichts der von Hamari et al. beschriebenen starken Kontextabhängigkeit von Gamifizierungseffekten ist das eine erhebliche Lücke [11], und sie lässt sich nur durch eine Erhebung mit tatsächlichen Anfängerinnen und Anfängern schließen.

6.3 Bekannte Schwächen der Umsetzung

Bei der Durchsicht des aktuellen Standes sind mehrere Punkte aufgefallen, die hier vollständig genannt werden sollen.

Die Zahl der Level ist an drei Stellen unabhängig voneinander hinterlegt. Die Routenerzeugung kennt dreizehn Level, zwei Konstanten in der Szene und in der Bestenliste stehen aber weiterhin auf neun. Die Bestenliste zeigt daher nur neun Spalten, und die Abschlussfeier für das Durchspielen löst nach Level 9 aus statt nach Level 13. Nichts im Aufbau erzwingt, dass diese Werte übereinstimmen, weshalb sie auseinandergefallen sind. Die richtige Behandlung ist, die Zahl aus dem vorhandenen Levelbestand abzuleiten, statt sie an drei Stellen zu schreiben.

Die Spalten für Schritte und Codezeilen wurden nachträglich ergänzt und sind deshalb in der Datenbank optional. Einträge, die vor dieser Erweiterung entstanden sind, tragen dort keinen Wert und werden über einen Ersatzwert ans Listenende sortiert. In Abb. 4 ist ein solcher Eintrag zu sehen. Solange die Datenbank Altbestand enthält, ist die Rangfolge damit nicht durchgängig vergleichbar.

Die Kennzahlen werden im Browser gezählt und vom Browser gemeldet. Der Dienst übernimmt sie ungeprüft. Wer den Worker verändert, kann beliebige Werte einreichen. Für den Einsatz in einer Lehrveranstaltung ist das vertretbar, für einen offen zugänglichen Wettbewerb nicht. Eine serverseitige Prüfung müsste den eingereichten Quelltext gegen das Level nachrechnen, was die bisher rein clientseitige Ausführung teilweise auf den Server zurückholen würde.

Die Bestenliste verlangt einen Betrieb, den das ursprüngliche Konzept ausdrücklich vermeiden wollte. Der Ausweg über die stille Abschaltung funktioniert, erzeugt aber zwei Betriebsarten, die beide gepflegt und getestet werden müssen. Dieser Aufwand taucht in keinem Funktionsumfang auf, fällt aber dauerhaft an.

Im Editor gebaute Level bleiben im `localStorage` des jeweiligen Browsers. Ein Austausch ist nur über den Umweg einer Datei oder eines GitHub-Issues möglich, und letzteres verlangt ein GitHub-Konto. Für die Zielgruppe ist diese Hürde deutlich höher als für die Projektbeteiligten, und sie steht im Widerspruch zu dem Anspruch, Autonomie zu ermöglichen.

6.4 Zum Risiko der Überrechtfertigung

Die von Deci et al. beschriebene Gefahr, dass eine eingeführte Belohnung die ursprüngliche Motivation verdrängt, lässt sich für dieses Projekt nicht ausschließen [12]. Es ist denkbar, dass jemand ein Level nicht mehr löst, um eine Schleife zu verstehen, sondern um in einer Tabelle weiter oben zu stehen.

Zwei Entwurfsentscheidungen wirken dem entgegen. Die Rangfolge nach Schritten belohnt genau das Verständnis, das ohnehin Lernziel ist, und lässt sich nicht durch Fleiß ersetzen. Und die Beschränkung auf die erste Lösung nimmt der Liste die Eigenschaft, durch bloße Wiederholung erklimmbar zu sein. Ob das ausreicht, ist ohne die fehlende Erhebung nicht zu beantworten.

7 Schlussfolgerung

7.1 Zusammenfassung

Am Anfang dieses Moduls stand eine Anwendung, die Python im Browser ausführen und eine Drohne animieren konnte, deren Ablauf aber mit dem Erreichen des Ziels endete. Am Ende steht eine Anwendung mit einer geschlossenen Schleife: Ein Level stellt eine Aufgabe, die Simulation zeigt die Folgen des geschriebenen Programms, eine Auswertung bewertet die Lösung nach Schritten, Abstürzen und Codezeilen, eine Bestenliste stellt sie neben die anderer, und ein Editor erlaubt es, die nächste Aufgabe selbst zu bauen.

Der Umfang der Arbeit lässt sich beziffern. In 50 Commits und elf Pull Requests wuchsen die spielbaren Level von sechs auf dreizehn, die Drohnenbefehle von zehn auf vierzehn und die Blocktypen von neun auf vierzehn. Hinzu kamen ein Backend-Dienst mit Kontoverwaltung, ein dreidimensionaler Editor und eine automatisierte Testabdeckung mit 118 Testfunktionen.

Die begriffliche Auswertung zeigt zugleich, wo der Entwurf schief steht. Kompetenzerleben ist gut bedient, Autonomie durch den Editor ebenfalls, die soziale Ebene bleibt bei einer Tabelle stehen. Und die zentrale Frage, ob die eingebauten Elemente bei tatsächlichen Anfängern die erhoffte Wirkung entfalten, ist mangels Erhebung offen.

7.2 Nächste Schritte

- **Nutzerstudie:** Erhebung mit Personen ohne Programmiererfahrung, mit einer Gruppe ohne und einer mit aktivierter Bestenliste, um die bislang nur begründeten Wirkungen zu prüfen.
- **Levelzahl ableiten:** Die drei getrennt gepflegten Angaben zur Zahl der Level durch eine gemeinsame Quelle ersetzen, damit Bestenliste und Abschlussfeier den tatsächlichen Bestand abbilden.
- **Prüfung der Kennzahlen:** Serverseitige Plausibilitätsprüfung eingereicherter Werte, mindestens gegen eine untere Schranke der Schrittzahl je Level.
- **Level teilen:** Ablage der im Editor gebauten Level im Backend, sodass sie sich ohne GitHub-Konto veröffentlichen und von anderen spielen lassen.
- **Abzeichen für Konzepte:** Auszeichnungen, die nicht an Geschwindigkeit hängen, sondern daran, ein Level erstmals mit einer Schleife, einer Funktion oder einer Bedingung gelöst zu haben. Sie besetzen den Teil der PBL-Trias, der bisher fehlt, ohne den Wettbewerbsdruck zu erhöhen.
- **Gemeinsame Aufgaben:** Wöchentlich wechselnde Level oder Aufgaben, die von mehreren gelöst werden, um die schwach besetzte soziale Ebene zu stärken.
- **Zweite Wertung:** Neben der gewerteten Erstlösung eine gesonderte Bestenliste für die beste jemals eingereichte Lösung, damit sich das Überarbeiten eines Programms wieder lohnt.

Literatur

- [1] „PythonWithDrones, Quelltext-Repository“. [Online]. Verfügbar unter: <https://github.com/pottmeier/PythonWithDrones>
- [2] „Pyodide Documentation“. [Online]. Verfügbar unter: <https://pyodide.org/>
- [3] „React Three Fiber Documentation“. [Online]. Verfügbar unter: <https://r3f.docs.pmnd.rs/>
- [4] S. Deterding, D. Dixon, R. Khaled, und L. Nacke, „From Game Design Elements to Gamefulness: Defining Gamification“, in *Proceedings of the 15th International Academic MindTrek Conference*, ACM, 2011, S. 9–15.
- [5] K. Werbach und D. Hunter, *For the Win: How Game Thinking Can Revolutionize Your Business*. Wharton Digital Press, 2012.
- [6] R. Hunicke, M. LeBlanc, und R. Zubek, „MDA: A Formal Approach to Game Design and Game Research“, in *Proceedings of the AAAI Workshop on Challenges in Game AI*, 2004.
- [7] R. M. Ryan und E. L. Deci, „Self-Determination Theory and the Facilitation of Intrinsic Motivation, Social Development, and Well-Being“, *American Psychologist*, Bd. 55, Nr. 1, S. 68–78, 2000.
- [8] M. Sailer, J. U. Hense, S. K. Mayr, und H. Mandl, „How Gamification Motivates: An Experimental Study of the Effects of Specific Game Design Elements on Psychological Need Satisfaction“, *Computers in Human Behavior*, Bd. 69, S. 371–380, 2017.
- [9] M. Csikszentmihalyi, *Flow: The Psychology of Optimal Experience*. Harper und Row, 1990.
- [10] I. Bogost, „Gamification is Bullshit“. [Online]. Verfügbar unter: https://bogost.com/writing/blog/gamification_is_bullshit/
- [11] J. Hamari, J. Koivisto, und H. Sarsa, „Does Gamification Work? A Literature Review of Empirical Studies on Gamification“, in *Proceedings of the 47th Hawaii International Conference on System Sciences*, 2014, S. 3025–3034.
- [12] E. L. Deci, R. Koestner, und R. M. Ryan, „A Meta-Analytic Review of Experiments Examining the Effects of Extrinsic Rewards on Intrinsic Motivation“, *Psychological Bulletin*, Bd. 125, Nr. 6, S. 627–668, 1999.
- [13] „FastAPI Documentation“. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/>
- [14] „SQLAlchemy Documentation“. [Online]. Verfügbar unter: <https://docs.sqlalchemy.org/>